



Real time community chats

Meta has announced a new feature called Community Chats that will soon see the light of day. <https://digit.in/oct22-59>



New plant for Intel

Intel has kicked off work on the new \$20 billion semiconductor plant in the Ohio state in US. <https://digit.in/oct22-60>

modern compilers are quite efficient at dealing with control flow conditions. Some compilers might unroll loop functions and use branch prediction to handle switch-case and if conditional statements. This ensures that there are no divergent warps being created.

You can also reduce the number of instructions by synchronising wherever possible. Another way to reduce instructions would be by using restricted pointers.

MINIMISE MEMORY THRASHING

In a distributed system, data can be saved onto any device and the changes that are done to mapped memory needs to be reflected everywhere. As more and more applications or processes or functions start modifying different portions of the memory, there is a lot of data synchronisation happening. This means, each new process or function has to wait for a data synchronisation to happen before it can access the memory. This leads to a little wait time. And as the number of processes or functions increase, this wait time scales quite a bit and you experience thrashing. There is also memory that's being released to be used by a completely independent external process which might be associated with the operating system. Over time, the allocation calls tend to get slower and slower and utilisation drops.

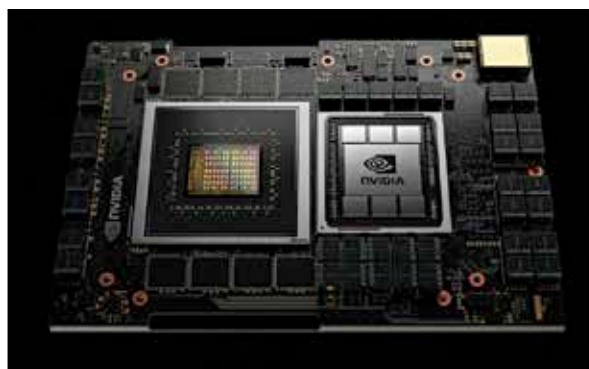
To deal with memory thrashing, you can allocate as much memory as

you believe will be required by the applications. However, the urge to allocate all the memory should be avoided. There are other applications which also need to use the memory and allocating the entire memory brings those processes to a halt.

And if the entire device memory gets allocated for a single application then the host memory needs to be used and that will slow things down considerably.

If possible, allocate memory as early as possible within the code and then avoid using `cudaMalloc()` or `cudaFree()` frequently. This is to be followed especially in functions where the performance is critical. If the workload is heavily parallelised, then having to run `cudaMalloc()` can lead to significant delays.

And if some application has gone ahead and allocated all the device memory, you should always factor in a fallback by using the host memory. Yes, the host memory is much slower than a local device memory but using it will at least allow the program to progress further than having to wait until memory is released.



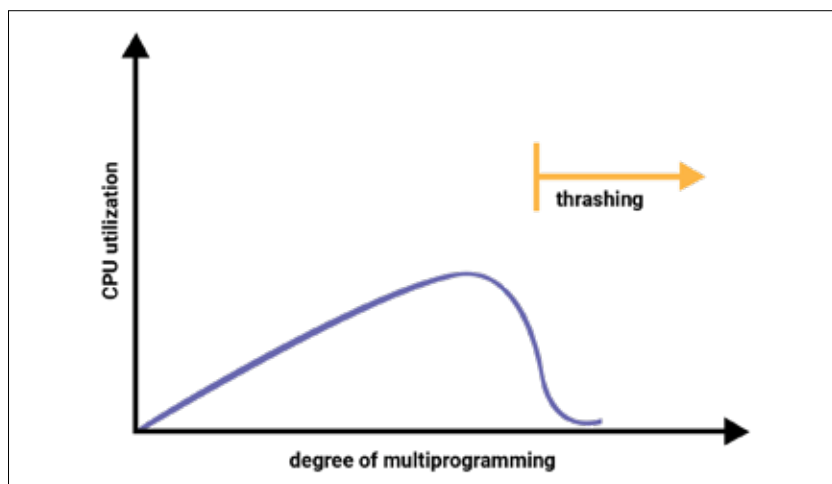
NVIDIA Grace Hopper with HBM memory

Certain platforms support `cudaMallocManaged`. This is a way to oversubscribe memory. This way, the memory allocation is not forced to be resident until and unless it is explicitly needed. Think of it like how airlines overbook passengers. While the passengers might not like it, the airline is always flying a packed flight.

LAST OPTION - UPGRADE HARDWARE

Even the most optimally written code cannot overcome the physical restrictions of not having enough Streaming Multiprocessors to play with. Also, if your code will benefit from much higher device memory, then the answer to getting a boost in performance is to upgrade your hardware. While the code written for CUDA will run on any NVIDIA graphics card that supports all the primitives used, there is the distinction between the consumer, enterprise workstation and data centre product lines.

The workstation and consumer cards top off at 48 and 24 GB of GDDR6X memory whereas the data centre card comes with 80 GB of HMB2e and HBM3 memory. HBM has a bus-width of 5120 bits which is magnitudes ahead of the 384-bit consumer cards. You can have much more parallel transfer using such a wide bus-width. So all things considered, optimised code will yield high returns but sooner or later, you might have to consider an upgrade as well. [▶](#)



Memory thrashing leads to reduced performance